PRIMER DEPARTAMENTAL- EJERCICIO 6- HERENCIA Y SOBREESCRITURA

En este programa desarrollamos una calculadora utilizando el paradigma de Programación Orientada a Objetos (POO), aplicando conceptos como herencia, encapsulamiento y polimorfismo. Creamos una clase padre llamada Calculadora, que contiene los atributos y métodos generales que comparten las operaciones. A partir de ella, creamos tres clases hijas: Multiplicacion, Potencia y Division, cada una encargada de realizar una operación diferente. De esta manera, aprovechamos la herencia para reutilizar código y utilizamos métodos sobrescritos para que cada clase ejecute su propia operación.

Código Completo:

```
#include <iostream>
using namespace std;

// CLASE PADRE
class Calculadora {
protected:
    long long num1, num2;

public:
    void capturar() {
        cout << "Ingresa el primer numero: ";
        cin >> num1;

        cout << "Ingresa el segundo numero: ";
        cin >> num2;
    }

    virtual void operar() {
        cout << "Operacion general" << endl;
    }
};

// CLASE HIJA: MULTIPLICACION
class Multiplicacion : public Calculadora {
public:
    void operar() {
        long long resultado = 0;
        long long a = num1;
        long long b = num2;

        if (b < 0) {
            a = -a;
            b = -b;
        }

        for (long long i = 0; i < b; i++) {
            resultado += a;
        }

        cout << "Resultado de la multiplicacion: "
             << resultado << endl;
    }
};

// CLASE HIJA: POTENCIA
```

```
class Potencia : public Calculadora {
public:
    void operar() {
        if (num2 < 0) {
            cout << "El exponente debe ser no negativo." << endl;
            return;
        }

        long long resultado = 1;

        for (long long i = 0; i < num2; i++) {
            long long temporal = 0;

            for (long long j = 0; j < resultado; j++) {
                temporal += num1;
            }

            resultado = temporal;
        }

        cout << "Resultado de la potencia: "
             << resultado << endl;
    }
};

// CLASE HIJA: DIVISION
class Division : public Calculadora {
public:
    void operar() {
        if (num2 == 0) {
            cout << "Error: no se puede dividir entre cero." << endl;
            return;
        }

        long long a = (num1 < 0) ? -num1 : num1;
        long long b = (num2 < 0) ? -num2 : num2;
        long long cociente = 0;

        while (a >= b) {
            a -= b;
            cociente++;
        }

        if ((num1 < 0 && num2 > 0) ||
            (num1 > 0 && num2 < 0)) {
            cociente = -cociente;
        }

        cout << "Cociente entero: " << cociente << endl;
        cout << "Residuo: " << a << endl;
    }
};

int main() {
    int opcion;

    Multiplicacion mult;
    Potencia pot;
    Division div;

    do {
        cout << "\n===== CALCULADORA =====" << endl;
        cout << "1. Multiplicacion" << endl;
```

```
        cout << "2. Potencia" << endl;
        cout << "3. Division" << endl;
        cout << "4. Salir" << endl;
        cout << "Elige una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                mult.capturar();
                mult.operar();
                break;

            case 2:
                pot.capturar();
                pot.operar();
                break;

            case 3:
                div.capturar();
                div.operar();
                break;

            case 4:
                cout << "Fin del programa." << endl;
                break;

            default:
                cout << "Opcion no valida." << endl;
        }

    } while (opcion != 4);

    return 0;
}
```

Explicación del Código:

1. Definición de la Clase Padre (class Calculadora)

Es la clase principal o plantilla de la que se derivan las demás clases. Contiene los atributos y métodos generales que necesitan todas las operaciones de la calculadora. En este caso, funciona como la base para las clases hijas Multiplicacion, Potencia y Division.

2. Declaración de Atributos Protegidos (protected:)

Dentro de esta sección declaramos las variables num1 y num2, que almacenan los dos números ingresados por el usuario. Utilizamos protected: para permitir que estos atributos puedan ser utilizados directamente tanto dentro de la clase padre como en sus clases hijas, sin necesidad de declararlos nuevamente.

3. Método para Capturar los Números (void capturar())

Es el método encargado de solicitar al usuario los dos números que se utilizarán en las operaciones. Se encuentra dentro de la clase padre Calculadora, por lo que las tres clases hijas pueden utilizarlo mediante la herencia, evitando repetir las mismas instrucciones en cada clase.

4. Método Virtual (virtual void operar())

Es un método declarado en la clase padre que puede ser redefinido por las clases hijas. Utilizamos la palabra reservada virtual para permitir el polimorfismo, es decir, que cada clase hija pueda tener su propia versión del método operar() y realizar una operación diferente.

5. Clase Hija de Multiplicación (class Multiplicacion : public Calculadora)

Esta clase hereda los atributos y métodos de la clase padre Calculadora mediante public. Su método operar() realiza la multiplicación utilizando sumas repetidas dentro de un ciclo for, en lugar de utilizar directamente el operador *. Si el segundo número es negativo, cambia los signos para obtener el resultado correcto.

6. Clase Hija de Potencia (class Potencia : public Calculadora)

Esta clase también hereda de Calculadora y se encarga de calcular la potencia de dos números. Su método operar() utiliza ciclos for anidados para realizar multiplicaciones mediante sumas repetidas. Primero verifica que el exponente no sea negativo y después calcula el resultado. La variable resultado comienza en 1, ya que cualquier número elevado a la potencia cero es igual a 1.

7. Clase Hija de División (class Division : public Calculadora)

Es la tercera clase derivada de Calculadora y se encarga de realizar la división. Su método operar() primero verifica que el divisor no sea cero. Después utiliza un ciclo while para realizar restas repetidas y obtener el cociente entero. También verifica los signos de los números para determinar si el resultado debe ser negativo y muestra el residuo de la división.

8. Instanciación de Objetos (Multiplicacion mult;, Potencia pot;, Division div;)

Dentro de la función main() creamos tres objetos, cada uno perteneciente a una clase diferente. El objeto mult permite realizar multiplicaciones, pot permite calcular potencias y div permite realizar divisiones. Cada objeto tiene acceso a los métodos heredados de la clase padre y a los métodos propios de su clase.

9. Función Principal (int main())

Es el punto de entrada del programa. Dentro de ella declaramos los objetos, mostramos el menú y controlamos las opciones mediante el switch y el ciclo do-while. Su función principal es coordinar el funcionamiento de las tres clases hijas y permitir que el usuario utilice la calculadora.